```
    }

    for (i=0;i<2;i++) {
      err = misc_register(&mydevice[i]);
      if(err < 0)
          goto error;
    }
    return;

error:
    for(--i;i>=0;i--)
      misc_deregister(&mydevice[i]);

err_unreg_device:
    parport_unregister_device(pprt);
    pprt = NULL;
}

static void mydevice_detach(struct parport *port)
{
    int i;

    if (port->number != 0)
      return;

    if (!pprt) {
      pr_err("nothing to unregister.\n");
      return;
    }

    for(i=0;i<2;i++)
      misc_deregister(&mydevice[i]);

    parport_release(pprt);
    parport_unregister_device(pprt);
    pprt = NULL;
}
```

Since we are using only two devices, we are registering two *miscdevices* here. If you plan to use more than two, you need to make the required modifications in the code. *mydevice*, which we have used with *misc_register*, is just an array of two elements defining the *miscdevice*. (If you have more than two devices, you need to add elements in this array of *mydevice*.)

```
static struct miscdevice mydevice[2] = {
    {
      .minor = MISC_DYNAMIC_MINOR,
      .name = "mydev1",
      .fops = &mydevice_fops,
      .nodename = "mydev1",
      .mode = 0666,  // pin 0 = STROBE
    },
    {
      .minor = MISC_DYNAMIC_MINOR,
      .name = "mydev2",
      .fops = &mydevice_fops,
      .nodename = "mydev2",
      .mode = 0666, // pin 5 = D3
    }
};
```

Instead of using the names *mydev1* or *mydev2*, you can easily use the names *fan* and *light*, in which case your device nodes will become */dev/fan* and */dev/light* but here, in this example, let's keep the nodes as */dev/mydev1* and */dev/mydev2:*

```
static const struct file_operations mydevice_fops = {
    .owner = THIS_MODULE,
    .write= mydevice_write,
    .open = mydevice_open,
    .release = mydevice_release,
};
```

This is the definition of *mydevice_fops* as we just need to open and write to the device. If you want to know the state of the device, you can also add a read call.

Now, we have a small problem here. We have created two device nodes and may choose to open one of them. Since we have a common open function for all the devices, how would we know which device was opened? Again, when you are writing to the device, how will the right function know which port pin to modify? Let's look at the open callback function to find out how we can solve this problem.

```
static int mydevice_open(struct inode *inode, struct file
*file)
{
    int i,minor = iminor(inode);
    int *deviceno = kmalloc(sizeof(int),GFP_KERNEL);
    for (i=0;i<2;i++) {
      dev_t dvt = (mydevice[i].this_device)->devt;
      if( minor == MINOR(dvt)) {
          *deviceno = i;
          break;
      }
    }

    file->private_data = deviceno;
    return 0;
}
```

The *struct* file has a member called *private_data* which is of type void *. You can store any kind of data there, which